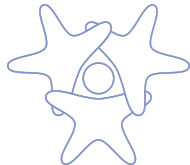




Disciplina: Engenharia de Software e Gestão de Times Ágeis

RUP e Desenvolvimento de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com



Objetivos da Aula

- Entender processos: Do cascata ao incremental, passando por espiral, reuso, RUP, Agile e DevOps.
- Conectar teoria e prática: Mostrar artefatos, papéis, fases, riscos, métricas e entregas executáveis.
- Apoiar estudo e docência: Gerar um roteiro de aula longo, progressivo e fácil de adaptar para graduação ou treinamento.



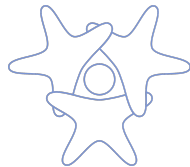
Objetivos da Aula

- Entender processos: Do cascata ao incremental, passando por espiral, reuso, RUP, Agile e DevOps.
- Conectar teoria e prática: Mostrar artefatos, papéis, fases, riscos, métricas e entregas executáveis.
- Apoiar estudo e docência: Gerar um roteiro de aula longo, progressivo e fácil de adaptar para graduação ou treinamento.



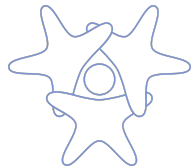
O que é um processo de software?

- Envolve entradas, saídas, responsáveis, dependências e critérios de execução.
- Ajuda a coordenar pessoas, artefatos, ferramentas e decisões.
- Não é burocracia por si só; é um mecanismo para aumentar previsibilidade e qualidade.



Modelo de processo

- Atividades
- O que precisa ser feito:
elicit
requisitos,
projetar,
codificar,
testar,
implantar.
- Agentes
- Papéis que executam as atividades:
analista,
arquiteto, dev,
tester, gerente,
PO.
- Artefatos
- Produtos gerados:
backlog, casos de uso,
diagramas,
código, testes,
releases,
métricas.



Processo, método e metodologia

- Processo
 - É a visão macro: fluxo de atividades, papéis, artefatos e ordem de execução.
 - Metodologia
 - Em muitos contextos, é o conjunto prático de métodos, princípios e ferramentas usado para especializar um processo em determinado ambiente organizacional.
- Método
 - É a forma sistemática de executar uma atividade específica, com passos e guias.



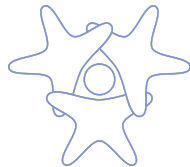
Por que processos importam?

- 1
 - 2
 - 3
 - 4
-
- Reduzem retrabalho.
 - Aumentam rastreabilidade.
 - Ajudam a gerenciar riscos.
 - Tornam a entrega mais previsível.



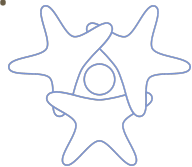
Modelo de ciclo de vida

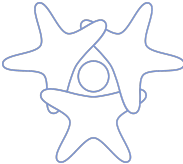
- É uma representação abstrata e simplificada de como o software nasce, evolui, é validado, implantado e mantido. O objetivo é organizar as principais atividades e os dados usados na produção e manutenção do sistema.
- Planejar
- Construir
- Evoluir



Principais modelos

- Sequenciais
- Cascata e variantes orientadas por fases.
- Evolucionários
- Programação exploratória, prototipagem e incremental.
- Iterativos
- Espiral, RUP, métodos ágeis e ciclos curtos de feedback.



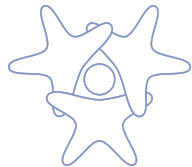


UNINASSAU
Campus Boa Viagem

Modelos clássicos e iterativos

Cascata: ideia central

- Requisitos, projeto, implementação, testes, operação e manutenção.
- Ajuda a enxergar o projeto como uma cadeia clara de produção.
- Reage mal a mudanças tardias e pressupõe alto nível de estabilidade dos requisitos.



Cascata em cinco macroetapas

Requisitos
Definir
necessidades.

Projeto
Estruturar
solução.

Implementação
Codificar
módulos.

Integração/Teste
Verificar o
sistema.

Operação
Usar e
manter.





Vantagens do cascata

Simplicidade

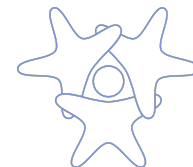
É fácil de ensinar, explicar e governar em ambientes regulados.

Documentação forte

Favorece formalização de decisões e contratos.

Marcos claros

Ajuda em auditoria, aprovação e acompanhamento gerencial.

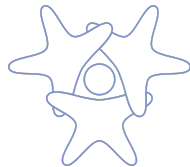


Problemas do cascata

O valor real do sistema é percebido muito tarde.

A descoberta tardia de falhas ou ambiguidades gera retrabalho elevado.

Integração, desempenho e usabilidade podem explodir apenas perto da entrega.



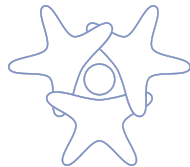


Programação exploratória

A primeira versão é produzida rapidamente, depois modificada sucessivamente conforme comentários dos usuários. Essa abordagem funciona quando é difícil obter uma especificação detalhada desde o início.

Rápida

Interativa





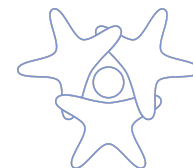
Quando a exploração faz sentido

Contextos favoráveis

Inovação, P&D, produtos com alto desconhecimento do domínio, interfaces novas e problemas mal estruturados.

Riscos associados

Arquitetura fraca, crescimento desordenado, pouca documentação e dependência de profissionais muito experientes.

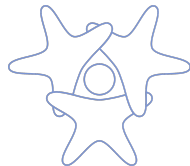


Prototipagem descartável

Entender requisitos, validar interface e demonstrar viabilidade.

O protótipo não é o produto final; ele existe para aprender e depois ser reimplementado.

Não tentar colocar todo o sistema no protótipo, senão o aprendizado perde foco.





Boas perguntas antes de prototipar

O que queremos
aprender?

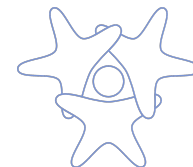
Requisitos, usabilidade, risco
técnico ou venda interna?

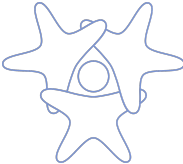
Quem valida?

Usuários, patrocinadores,
área de negócio e equipe
técnica.

O que não entra?

Escopo explícito evita
desperdício e interpretações
perigosas.





UNINASSAU
Campus Boa Viagem

Espiral, incremental e reuso



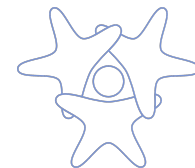
Desenvolvimento baseado em reuso

Ideia

Usar componentes existentes, bibliotecas, frameworks ou sistemas COTS para acelerar a entrega.

Efeito

Muda requisitos, arquitetura e integração; desenvolver nem sempre significa construir tudo do zero.





Etapas do processo com reuso

Requisitos

Definir necessidade.

Buscar componentes

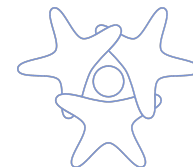
Analisar mercado e
catálogo interno.

Ajustar solução

Revisar requisitos e
arquitetura.

Integrar e validar

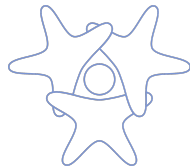
Montar, testar e
confirmar aderência.





Por que iterar?

Grandes sistemas sempre mudam durante o projeto. A iteração reconhece essa realidade e organiza o trabalho em ciclos que permitem revisar entendimento, reduzir risco e entregar partes utilizáveis do produto.



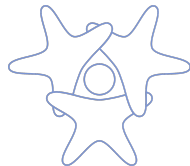


Modelo espiral

Cada volta da espiral avalia alternativas, riscos e decisões.

O foco depende do que o projeto exige naquele momento.

Planejamento, controle e validação aparecem desde cedo.



Quatro movimentos típicos da espiral

Objetivos

Definir metas e restrições.

Riscos

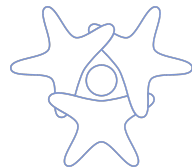
Analisar alternativas e mitigação.

Engenharia

Construir e validar versão atual.

Plano

Preparar próximo ciclo.





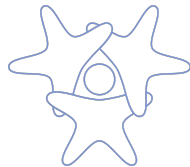
Quando usar espiral

Bom cenário

Projetos grandes, caros, críticos e com alto grau de incerteza técnica ou de negócio.

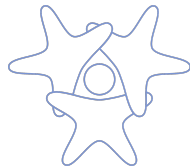
Atenção

Exige maturidade em gestão, análise de risco e tomada de decisão contínua.



Desenvolvimento incremental

Em vez de entregar tudo de uma vez, o sistema é dividido em incrementos. As funcionalidades de maior prioridade entram primeiro, e cada incremento produz parte do valor esperado pelo usuário.



Fluxo típico do incremental

1. Esboçar requisitos

Definir o escopo global.

2. Priorizar e distribuir

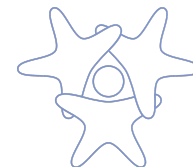
Associar requisitos a incrementos.

3. Projetar arquitetura

Criar base estável.

4. Desenvolver e validar

Entregar incremento executável.





Benefícios do incremental

Valor cedo

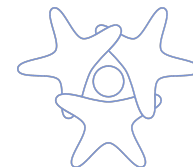
Usuário vê resultados antes do fim do projeto.

Risco diluído

Problemas aparecem em lotes menores e mais controláveis.

Aprendizado contínuo

Cada entrega melhora a próxima iteração.





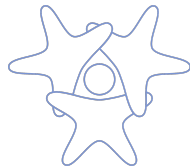
UNINASSAU
Campus Boa Viagem

Visão geral do RUP



O que é o RUP?

O nome significa Rational Unified Process. Na prática, ele combina processo, métodos, guias e uso de UML para organizar o desenvolvimento.



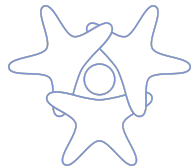
RUP como framework

Ideia principal

Mais do que um roteiro fixo, o RUP foi concebido como um framework configurável para gerar processos adaptados ao contexto do projeto.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Características centrais

Definição

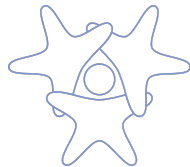
O RUP é iterativo e incremental, guiado por casos de uso e centrado na arquitetura do sistema.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

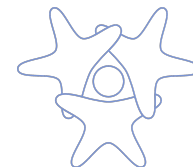


Quatro fases do ciclo

Concepção, elaboração, construção e transição compõem o ciclo de vida do RUP.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Concepção

01

Definir

02

Validar

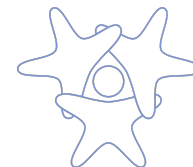
03

Executar

04

Aprender

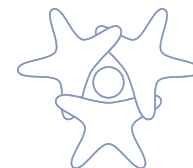
Define escopo, visão, viabilidade, principais requisitos, estimativas iniciais e riscos principais.





Elaboração

Detalha requisitos prioritários e estabiliza a arquitetura executável do sistema.





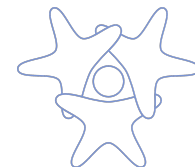
Construção

Ideia principal

Implementa a maior parte das funcionalidades, amplia qualidade e prepara o produto para uso real.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Transição

Definição

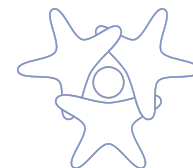
Implanta o sistema, treina usuários, corrige defeitos finais e faz a passagem ao ambiente operacional.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



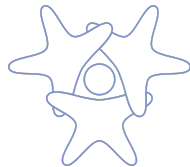


Milestones no RUP

Cada fase termina com um marco importante para decidir se o projeto avança com segurança.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Iterações dentro das fases

01

Definir

02

Validar

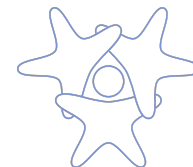
03

Executar

04

Aprender

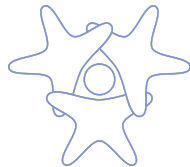
Toda fase possui uma ou mais iterações planejadas, cada uma com objetivos, atividades e critérios de sucesso.





Entregas executáveis

Em geral, cada iteração produz uma versão executável ou demonstrável do sistema.



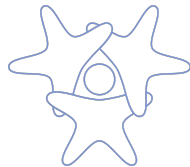
RUP e casos de uso

Ideia principal

Casos de uso orientam requisitos, planejamento de iterações, projeto, integração e testes.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





RUP e arquitetura

Definição

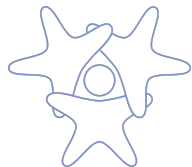
A arquitetura é definida cedo e usada como base para crescimento controlado do sistema.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

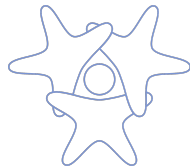


Papéis no RUP

O processo define workers ou papéis, não pessoas específicas: analista, arquiteto, gerente, desenvolvedor, tester etc.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Artefatos no RUP

01

Definir

02

Validar

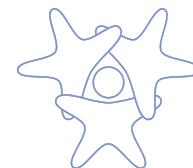
03

Executar

04

Aprender

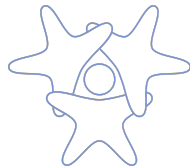
Cada atividade tem entradas, saídas, templates, guias e responsabilidades ligadas a artefatos concretos.





Fluxos principais

Modelagem de negócio, requisitos, análise e projeto, implementação, teste e implantação.





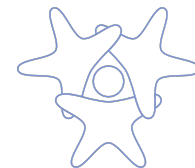
Fluxos de apoio

Ideia principal

Gerência de configuração e mudanças, gerência de projeto e ambiente.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Concepção em detalhe

Definição

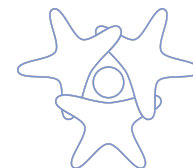
Aqui se cria visão do produto, business case, escopo inicial e avaliação dos riscos de maior impacto.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

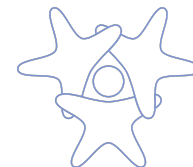


Perguntas da concepção

Vale a pena fazer? Para quem? Qual problema resolve? Quais riscos ameaçam a iniciativa?

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Artefatos da concepção

01

Definir

02

Validar

03

Executar

04

Aprender

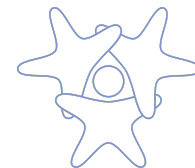
Visão, caso de negócio, lista de riscos, glossário inicial, plano preliminar e casos de uso principais.





Elaboração em detalhe

A meta é eliminar riscos críticos e obter uma baseline arquitetural executável.





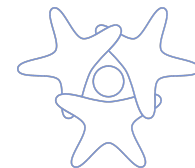
Arquitetura executável

Ideia principal

Não basta um desenho bonito; a arquitetura precisa ser prototipada e validada com execução real.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Riscos arquiteturais

Definição

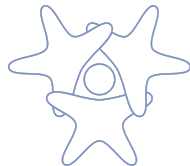
Integração, desempenho, segurança, dados, escalabilidade e dependências externas devem ser atacados cedo.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

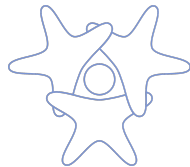


Artefatos da elaboração

Modelo de casos de uso mais completo, arquitetura, protótipos, plano refinado e critérios de construção.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Construção em detalhe

01

Definir

02

Validar

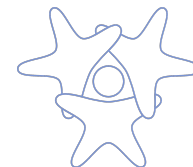
03

Executar

04

Aprender

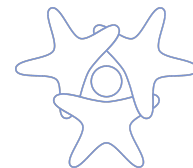
Nesta fase o esforço se volta ao desenvolvimento da funcionalidade, integração contínua e qualidade do produto.





Construção não é codar sem pensar

Ainda existe análise, design, testes e revisão; só o foco relativo muda em direção à implementação.





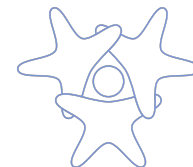
Transição em detalhe

Ideia principal

Inclui deploy, migração de dados, suporte inicial, treinamento e ajustes finais de aceitação.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Critério de sucesso da transição

Definição

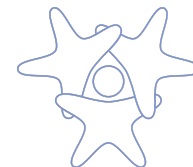
O sistema é aceito, usado pelo público-alvo e atende qualidade mínima de operação.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



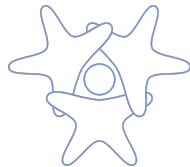


RUP é iterativo

Cada iteração repete atividades de requisitos, design, implementação e teste em escala menor.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





RUP é incremental

01

Definir

02

Validar

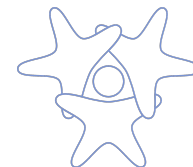
03

Executar

04

Aprender

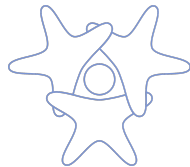
Cada ciclo agrega partes novas ao produto, em vez de reiniciar do zero.





Vantagem pedagógica do RUP

Ele ajuda estudantes a enxergarem a ligação entre artefatos, papéis, arquitetura, risco e entrega.





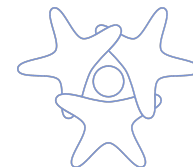
Vantagem gerencial do RUP

Ideia principal

Permite visibilidade maior do andamento e checkpoints formais entre fases e iterações.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



RUP x Cascata

Definição

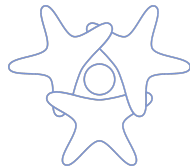
As fases existem, mas dentro delas há iteração, feedback e validação progressiva, não linearidade absoluta.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



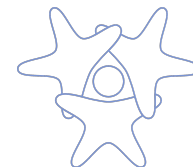


RUP x Agile

RUP tende a ser mais prescritivo e documentado; Agile costuma priorizar leveza, adaptabilidade e comunicação direta.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





RUP e disciplina

01

Definir

02

Validar

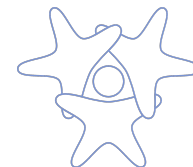
03

Executar

04

Aprender

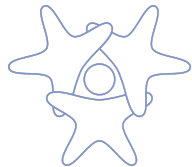
O famoso dilema é equilibrar agilidade e disciplina em vez de tratar os dois como opostos absolutos.





Casos de uso: conceito

Descrevem interações entre atores e sistema para atingir objetivos observáveis do negócio.

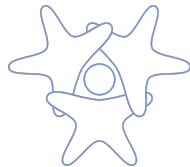


Ideia principal

Podem ser pessoas, sistemas externos, dispositivos ou papéis organizacionais que interagem com o software.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Estrutura de caso de uso

Definição

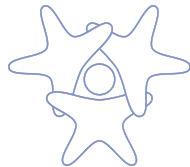
Nome, objetivo, atores, fluxo principal, fluxos alternativos, pré-condições e pós-condições.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

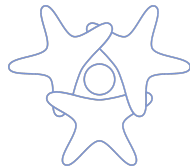


Casos de uso no planejamento

No RUP, eles ajudam a decidir ordem das iterações e prioridades de implementação.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Casos de uso nos testes

01

Definir

02

Validar

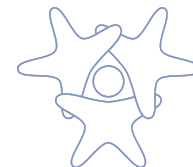
03

Executar

04

Aprender

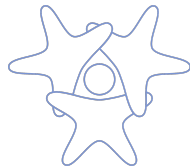
Cada cenário importante pode originar testes de aceitação e casos de teste detalhados.





Arquitetura de software

É a visão de alto nível do sistema, seus subsistemas, componentes e relacionamentos.





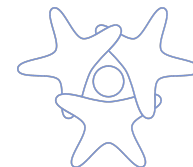
Por que arquitetura importa

Ideia principal

Ela organiza o time, facilita reuso e reduz risco técnico em sistemas complexos.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Visões arquiteturais

Definição

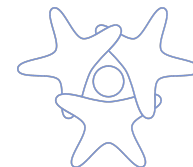
Lógica, implementação, processo, implantação e cenários são formas clássicas de descrever arquitetura.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

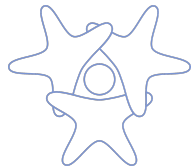


Arquitetura e qualidade

Decisões arquiteturais afetam desempenho, segurança, disponibilidade, escalabilidade e manutenção.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Requisitos funcionais

01

Definir

02

Validar

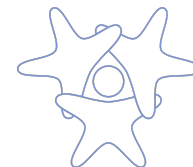
03

Executar

04

Aprender

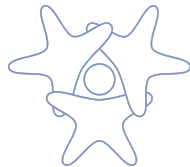
Descrevem o que o sistema deve fazer: regras, operações, fluxos e respostas esperadas.





Requisitos não funcionais

Descrevem restrições e qualidades: segurança, usabilidade, desempenho, conformidade, confiabilidade.





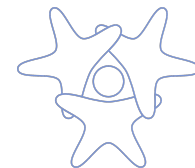
Engenharia de requisitos

Ideia principal

Envolve elicitar, analisar, especificar, validar e gerenciar mudanças ao longo do projeto.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Técnicas de elicitação

Definição

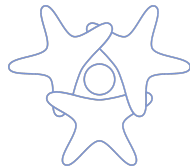
Entrevistas, workshops, observação, análise documental, prototipação e questionários.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



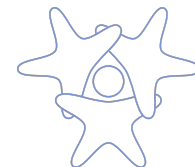


Problemas comuns de requisitos

Ambiguidade, omissão, conflito, volatilidade e linguagem excessivamente genérica.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Boas práticas de requisitos

01

Definir

02

Validar

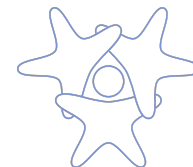
03

Executar

04

Aprender

Priorizar, tornar testável, manter rastreabilidade e validar cedo com stakeholders.





UML: visão geral

A UML é uma linguagem de modelagem amplamente usada para representar estrutura e comportamento de sistemas.

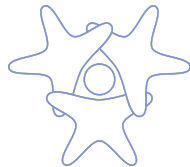


Diagrama de casos de uso

Ideia principal

Mostra atores, casos de uso e relações de forma resumida para comunicação de escopo.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.

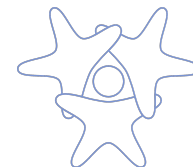


Diagrama de classes

Definição

Mostra classes, atributos, operações e relacionamentos como associação e herança.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

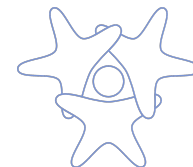
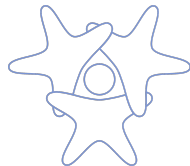


Diagrama de sequência

Mostra a ordem das mensagens entre objetos para realizar um cenário específico.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Outros diagramas úteis

01

Definir

02

Validar

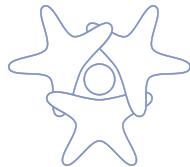
03

Executar

04

Aprender

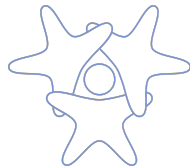
Atividades, estados, componentes e implantação também ajudam em análise e arquitetura.





Modelagem não é fim em si

Diagramas devem esclarecer decisões e comunicação, não virar burocracia ornamental.





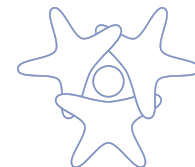
Análise e projeto

Ideia principal

Na análise, entendemos o problema; no projeto, definimos a solução técnica.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Do requisito ao código

Definição

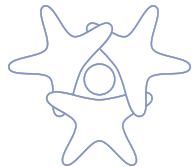
Uma cadeia saudável liga necessidade de negócio, modelo, arquitetura, implementação e teste.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



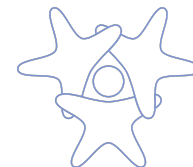


Padrões de projeto

Design patterns ajudam a resolver problemas recorrentes de forma mais consistente.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Coesão e acoplamento

01

Definir

02

Validar

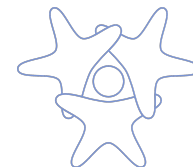
03

Executar

04

Aprender

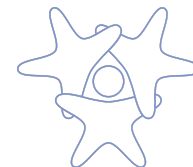
Boa arquitetura busca alta coesão e baixo acoplamento para facilitar evolução.





Implementação

Codificar envolve padrões, revisão, automação, integração contínua e disciplina técnica.





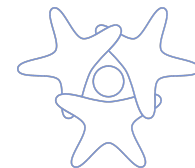
Integração contínua

Ideia principal

Mudanças frequentes no código são integradas e verificadas automaticamente para reduzir surpresas.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Estratégias de branching

Definição

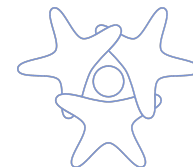
Trunk-based, GitFlow e variações precisam combinar com o ritmo de entrega e governança.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



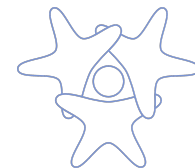


Qualidade de código

Legibilidade, testabilidade, simplicidade e consistência importam tanto quanto funcionalidade.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Testes unitários

01

Definir

02

Validar

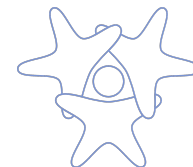
03

Executar

04

Aprender

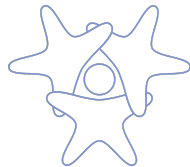
Validam pequenas unidades de código e ajudam a detectar regressões cedo.





Testes de integração

Avaliam interfaces entre módulos, serviços, banco de dados e sistemas externos.





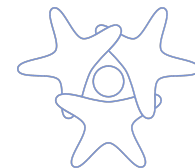
Testes de sistema

Ideia principal

Verificam comportamento do produto integrado em relação aos requisitos definidos.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Testes de aceitação

Definição

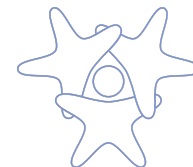
Confirmam se o software atende à expectativa de negócio e do usuário final.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



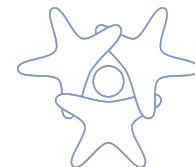


Pirâmide de testes

Muitos testes pequenos, menos testes integrados e poucos testes fim a fim tende a gerar equilíbrio.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Automação de testes

01

Definir

02

Validar

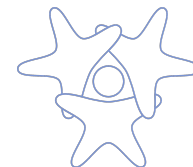
03

Executar

04

Aprender

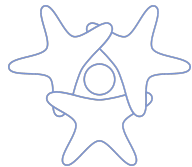
Aumenta velocidade de feedback e segurança para evoluir o produto continuamente.





Gerência de configuração

Controla versões de artefatos e garante integridade do produto ao longo do tempo.





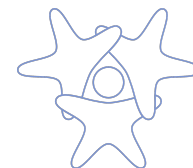
Gestão de mudanças

Ideia principal

Toda mudança relevante precisa ser registrada, analisada, priorizada e acompanhada.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



Gestão de projeto

Definição

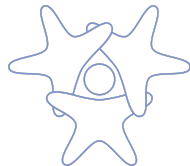
Escopo, prazo, custo, qualidade, riscos, comunicação e pessoas compõem o núcleo da gestão.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

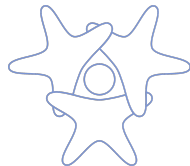


Riscos em software

Podem ser técnicos, organizacionais, financeiros, de cronograma, legais ou humanos.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Análise de risco

01

Definir

02

Validar

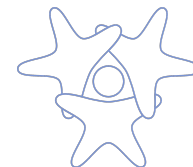
03

Executar

04

Aprender

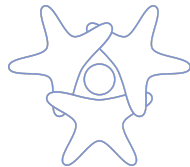
Probabilidade e impacto ajudam a priorizar mitigação, contingência e monitoramento.





Métricas de software

Velocidade, lead time, cobertura, defeitos e satisfação ajudam a enxergar desempenho.





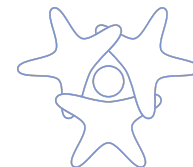
Cuidado com métricas

Ideia principal

Métrica ruim gera comportamento ruim; medir só produtividade pode incentivar dívida técnica.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





Documentação útil

Definição

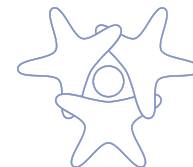
Boa documentação reduz dependência de memória oral e acelera onboarding e manutenção.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.

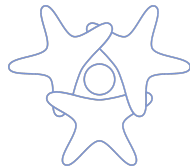


Arquitetura viva

Diagramas e registros de decisão precisam evoluir junto com o sistema.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Agile: princípios gerais

01

Definir

02

Validar

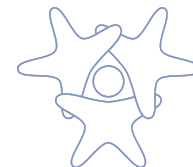
03

Executar

04

Aprender

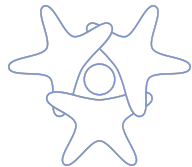
Pequenos ciclos, colaboração, adaptação e foco em software funcionando.





Scrum em resumo

Papéis, eventos e artefatos organizam trabalho iterativo em sprints com inspeção e adaptação.



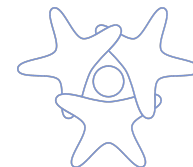
XP em resumo

Ideia principal

Valoriza práticas de engenharia como TDD, refatoração, integração contínua e programação em par.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.





RUP e Agile podem coexistir

Definição

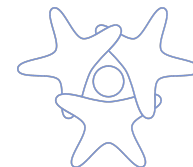
É possível ter governança inspirada em RUP com execução ágil e enxuta nas equipes.

Valor

Ajuda a reduzir incerteza e melhorar qualidade das decisões.

Risco

Quando ignorado, gera retrabalho, ambiguidade e decisões frágeis.



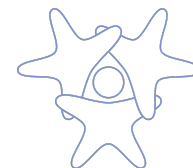


DevOps: visão geral

Integra desenvolvimento e operações em um fluxo contínuo de planejar, codificar, construir, testar, liberar, implantar, operar e monitorar.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Pipeline moderno

01

Definir

02

Validar

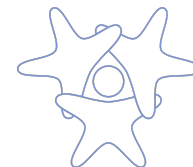
03

Executar

04

Aprender

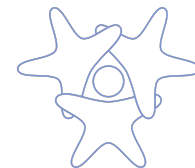
CI/CD reduz tempo entre mudança e entrega, mantendo controle com testes e automação.





Observabilidade

Logs, métricas e traces ajudam a entender comportamento real do sistema em produção.





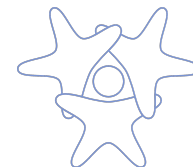
Segurança no ciclo

Ideia principal

DevSecOps traz práticas de segurança para o fluxo contínuo, não apenas para o final.

Aplicação prática

Use este conceito para orientar decisões de processo, artefatos, revisão e comunicação ao longo do projeto.



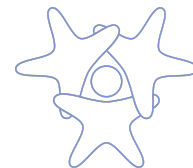


Finalmente!

Não existe processo perfeito; existe processo coerente com contexto, risco, equipe, produto e organização.

O time precisa transformar esse princípio em rotina, decisão arquitetural ou critério de qualidade.

É um bom ponto para discussão, estudo de caso ou exercício aplicado.





Disciplina: Engenharia de Software e Gestão de Times Ágeis

RUP e Desenvolvimento de Software

Prof. Pedro Rodolfo Gomes de Souza
prgsouza3@gmail.com

